

Use of Ridge Regression to Combine Focus Metrics in Inline Holographic Microscopy

Arda Aydinlar

May 2026

Abstract

In inline holographic microscopy, the correct numerical refocus distance must be determined computationally, as the sample-to-camera distance is unknown. Seven focus metrics are computed at 180 depth values across 676 holograms spanning five sample types: bee wing, ipomoea leaf, lillium anther, paramecium stained, and a USAF resolution target. Used individually, these metrics achieve a maximum fraction correct of 0.675 at a tolerance of 0.1 mm (`sobel_variance`). This work investigates whether machine learning can combine these seven metrics into a single, more reliable focus measure. Two models are trained and evaluated: Ridge regression, which learns a linear weighted combination, and a Multi-Layer Perceptron (MLP), which captures non-linear interactions between metrics. The Ridge model achieves a fraction correct of 0.676, a marginal improvement over the best individual metric. The MLP achieves 0.757, an improvement of 8.2 percentage points over the best individual metric. Both models achieve perfect performance on lillium anther samples but struggle with paramecium stained, where Ridge achieves only 0.250 compared to the MLP's 0.607. The superior performance of the MLP suggests that the relationship between focus metrics is not purely linear. This effect is most evident for paramecium stained samples, where the MLP substantially outperforms Ridge regression.

1 The Big Picture

The physical problem is autofocus in inline holographic microscopy: given a hologram of an unknown sample, find the correct numerical refocus distance. Seven different focus metrics are computed at 200 depth values for each hologram. The original dataset contains 750 holograms, of which 676 remain after applying the depth-offset filtering described in Section 4, spanning five biological and optical sample types. Each metric provides a partial, noisy signal about which depth is in focus, and no single metric reliably identifies the correct depth across all sample types and conditions.

Success is defined using the `fraction_correct` metric: the proportion of test images for which the model's predicted best-focus depth falls within a tolerance window of the manually-determined ground truth depth. The primary tolerance used is 0.1 mm, which corresponds to ten depth steps of 10 microns each. A secondary evaluation is also performed at tolerances of 0.05, 0.2, and 0.5 mm to give a fuller picture of model precision. The primary goal is to determine whether a machine-learning model can outperform the best individual focus metric.

2 Get Data

Dataset is provided as a Python pickle file (`focus_score_curves_dataset.pkl`)[2] and is loaded using the `load_focus_data` helper function from `helpers.py`. [1] The dataset contains:

- **scores:** An array of shape (750 images, 200 depths, 7 metrics) containing the raw focus metric values.
- **depths:** The 200 numerical refocus distances in millimetres, spanning 0 to 2 mm in 10 micron steps.
- **true_depths:** The manually-determined ground truth focus depth for each image (in mm).
- **image_types:** The sample type label for each image (paramecium, liliun, bee wing, ipomoea, or USAF target).
- **metrics:** The names of the seven focus metrics.

The seven metrics can be broadly grouped by what image property they measure. `sobel`, `sobel_variance`, `brenner`, and `dark_focus` are gradient-based metrics that respond to sharp edges and high-frequency image content. `norm_var` measures the normalised variance of pixel intensities. `sum` sums pixel values across the image, and `peak` records the maximum pixel value. Further details are available in the PyHoloscope documentation. [1] [4]

A pristine test set is isolated before any preprocessing or model fitting. The data is split at the image level and all 200 depth values for a given image go entirely into either the training set or the test set, never both. This is critical: splitting at the depth level would allow the same image to appear in both sets, causing data leakage and falsely inflating test performance. 80% of images are used for training (540 images) and 20% for testing (136 images). Stratification by `image_types` ensures each sample type is proportionally represented in both sets, so the test set fairly represents all conditions the model will encounter. [3]

Listing 1: Snippet of code from `ridge_regression.py` [2]

```
train_idx, test_idx = train_test_split(
    image_indices, test_size=0.2, random_state=42, stratify=data.
    image_types)
```

3 Explore

Before any model is trained, a baseline is established by evaluating each of the seven raw metrics individually using `fraction_correct` across four tolerance levels. The results are summarised in Table 1.

The results reveal a clear split among the metrics. The gradient-based metrics — `sobel` (0.669), `sobel_variance` (0.675), `brenner` (0.626), and `dark_focus` (0.580) — substantially outperform the remaining three at all tolerances. The `sum` and `norm_var` metrics perform at essentially random levels (0.059 at 0.1 mm), providing no useful signal when used individually under the `argmin` rule. The `peak` metric achieves a modest 0.228. The best individual metric, `sobel_variance`, achieves

Table 1: Fraction correct for each individual focus metric at four tolerance levels across all 676 images. The best-performing metric at each tolerance is shown in bold.

Metric	0.05 mm	0.10 mm	0.20 mm	0.50 mm
<code>sum</code>	0.015	0.059	0.133	0.396
<code>peak</code>	0.135	0.228	0.327	0.580
<code>sobel</code>	0.580	0.669	0.694	0.772
<code>sobel_variance</code>	0.604	0.675	0.743	0.830
<code>brenner</code>	0.558	0.626	0.664	0.771
<code>dark_focus</code>	0.512	0.580	0.624	0.725
<code>norm_var</code>	0.015	0.059	0.133	0.396

0.675 at 0.1 mm and consistently leads at all four tolerances; this becomes the target for any combined model to surpass.

Focus score curves for individual images are visualised using the `show_curves` helper function,[1] which plots all seven metric curves against depth and marks the ground truth depth with a vertical line. This exploration reveals several important properties of the data:

- All metrics produce noisy, unreliable outputs at distances below approximately 0.2 mm, because holographic reconstruction is poor at very short range. This motivates the depth offset applied in Section 4.
- Different sample types produce markedly different curve shapes. The USAF resolution target and lillium anther exhibit clean, well-defined minima, while biological samples such as paramecium stained produce noisier, flatter curves where the true minimum is ambiguous.
- For many images, multiple metrics simultaneously show local minima near the true depth, suggesting that a learned combination could outperform any single metric. This can be seen in Figures 7 and 8.

4 Prepare

Depth offset: The first 20 depth values (covering 0 to 0.2 mm) are removed by passing `offset=20` to `load_focus_data`. In this region, all metrics produce noisy, unreliable outputs because holograms do not reconstruct well at very short distances. Removing this region improves both training stability and evaluation accuracy. This reduces the depth range from 200 to 180 values per image. This filtering caused the dataset to change from 750 to 676 holograms and these were used for all the subsequent training and evaluation which can be seen in Table 2.

Stage	Number of holograms
Original dataset	750
Excluded (focus within 0.2 mm of boundary)	74
Final dataset used	676

Table 2: Dataset size after preprocessing and filtering.

Per-image normalisation: Each of the seven metrics is independently normalised to the range $[0, 1]$ within each image:

Listing 2: Snippet of code from *ridge_regression.py* [2]

```
s_min = scores.min(axis=1, keepdims=True)
s_max = scores.max(axis=1, keepdims=True)
scores_norm = (scores - s_min) / (s_max - s_min + 1e-8)
```

This is done per-image rather than globally because the absolute values of metrics vary considerably between images depending on sample brightness and contrast. Per-image normalisation ensures the model responds to the shape of the curve where the metric is low or high relative to its own range, rather than its absolute magnitude. The small epsilon ($1e-8$) prevents division by zero in edge cases.

Gaussian target labels: Ridge regression requires a scalar target value for every (image, depth) data point. A Gaussian-shaped soft label is constructed for each image, centred on its true depth: [2]

$$t(d) = \exp\left(-\frac{1}{2}\left(\frac{d - d_{\text{true}}}{\sigma}\right)^2\right) \quad (1)$$

This assigns a target value of 1.0 at the true focus depth, falling smoothly to 0.0 at depths far from focus. A sigma of 0.1 mm is used, which is wide enough for the model to learn from neighbouring depths but narrow enough to produce a precise minimum. The result is a well-posed regression problem: the model learns to predict high values near focus and low values far from it.

StandardScaler: After per-image normalisation, a StandardScaler from sklearn is applied to standardise each of the seven features to zero mean and unit variance across the training set. This is important for Ridge regression because the L2 penalty treats all features equally without standardisation, a feature with a slightly larger scale would be penalised more heavily than intended. The scaler is fit only on the training set and applied (without refitting) to the test set to prevent data leakage. [8]

Dataset construction: The data is flattened from shape (N_images, 180 depths, 7 metrics) into shape (N_images 180, 7) for sklearn compatibility, with corresponding Gaussian targets of shape (N_images 180,). This produces approximately 108,000 training samples.

5 Train

Ridge Regression

A Ridge regression model is trained using sklearn’s **Ridge** class with regularisation strength **alpha** = 1.0:[7]

Listing 3: Snippet from *ridge_regression.py* [2]

```
model = Ridge(alpha=1.0)
model.fit(X_train_scaled, y_train)
```

Ridge regression finds the weight vector **w** and bias **b** that minimise:

$$\mathcal{L}_{\text{Ridge}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 + \alpha \|\mathbf{w}\|^2 \quad (2)$$

The first term is the mean squared error between the predicted and Gaussian target values. The second term is the L2 penalty: α times the sum of squared weights. This penalty discourages any single metric from receiving a disproportionately large weight, which is important given that the seven metrics measure overlapping properties of the same image. Unlike the MLP described below, Ridge regression has a closed-form analytical solution — sklearn solves Equation 2 in a single linear algebra step, making training instantaneous and fully deterministic.[7]

Multi-Layer Perceptron

A Multi-Layer Perceptron (MLP) is trained as a more flexible alternative, capable of capturing non-linear interactions between metrics. The network is intentionally small because only seven input features are available and the dataset size is modest. Larger networks were unlikely to provide substantial benefit and would increase the risk of overfitting. The architecture consists of three fully connected layers:

- **Input layer:** 7 neurons, one per focus metric at a single depth
- **Hidden layer 1:** 32 neurons with ReLU activation
- **Hidden layer 2:** 16 neurons with ReLU activation
- **Output layer:** 1 neuron, producing a single focus score for that depth

The network is applied independently to each (image, depth) pair, treating depth as a simple index rather than a continuous input variable. Rather than Gaussian target labels, the MLP is trained using a margin ranking loss. For each image, the loss penalises any configuration in which the predicted score at the true focus depth is not sufficiently lower than all other depths:[6]

$$\mathcal{L}_{\text{MLP}} = \frac{1}{N} \sum_{i=1}^N \frac{1}{D} \sum_{d=1}^D \max\left(0, m + s_i^{(d_{\text{true}})} - s_i^{(d)}\right) \quad (3)$$

where $s_i^{(d)}$ is the model’s predicted score for image i at depth d , d_{true} is the true focus depth index, and $m = 0.2$ is the margin. The $\max(0, \cdot)$ clamp means the loss is zero when the true depth already scores lower than depth d by at least m , and positive only when this condition is violated.

The model is trained for 150 epochs using the Adam optimiser with learning rate 10^{-3} [5]. The training loss decreases smoothly from 0.2587 at epoch 0 to 0.0353 at epoch 135, confirming stable convergence throughout training (Figure 1). No sudden jumps or plateauing are observed, indicating that the learning rate and architecture are well-suited to the problem.

6 Fine-Tune

Ridge: regularisation strength. The key tuning parameter for Ridge regression is α in Equation 2. A sensitivity study was performed by retraining the model with five values of alpha and evaluating performance on the held-out test set in Table 3.

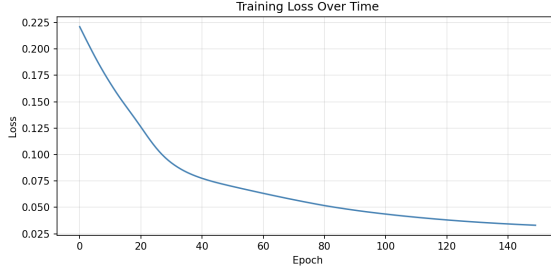


Figure 1: Training loss over epochs.

Table 3: Ridge regression fraction correct at `tol = 0.1mm` for five values of the regularisation parameter α .

α	Fraction correct (0.10 mm)
0.01	0.676
0.1	0.676
1.0	0.676
10.0	0.669
100.0	0.654

Performance is flat and equal across $\alpha \in \{0.01, 0.1, 1.0\}$, then degrades monotonically at higher values. This indicates the model is insensitive to the exact level of regularisation over a broad range but deteriorates when the penalty is strong enough to suppress the largest weights ($\alpha = 100.0$ reduces the dominant `norm_var` weight from $+0.788$ to $+0.153$, effectively discarding the signal it carries). The value `alpha = 1.0` is used for the final model as it is the most interpretable choice within the optimal range.

Ridge: Gaussian width. The σ parameter of the Gaussian target is set to 0.1 mm, corresponding to ten depth steps. This ensures that the immediate neighbourhood of the true depth also receives high target values, reducing sensitivity to the discrete sampling of the depth axis. A smaller σ (e.g. 0.05 mm) would create sharper, more demanding targets; a larger σ (e.g. 0.2 mm) would be easier to fit but may produce broader output curves with less precise minima.

MLP: margin and learning rate. The margin $m = 0.2$ in Equation 3 is set to twice the primary evaluation tolerance of 0.1 mm, so the model is trained to separate focus from non-focus by a margin that exceeds the measurement precision. The learning rate of 10^{-3} is the standard default for Adam and is consistent with the smooth convergence observed in training, where the loss decreased from 0.2587 at epoch 0 to 0.0353 at epoch 135.

Depth offset. Both models use `offset = 20`, removing the first 20 depth steps (0 to 0.2 mm). This is motivated by the data exploration in Section 3, where all seven metrics were observed to produce unreliable outputs in this region.

A systematic search over σ and the MLP margin using cross-validation on the training set would be a natural extension of this work, as would training separate models per sample type — particularly for paramecium stained, which both models handle poorly.

7 Present

Overall performance. Table 4 summarises the fraction correct at four tolerance levels for both models, compared against `sobel_variance` as the best individual metric.

Table 4: Fraction correct at four tolerance levels. The best individual metric is `sobel_variance`. Bold indicates the best result at each tolerance.

Model	0.05 mm	0.10 mm	0.20 mm	0.50 mm
<code>sobel_variance</code> (best individual)	0.604	0.675	0.743	0.830
Ridge regression ($\alpha = 1.0$)	0.566	0.676	0.735	0.831
MLP	0.654	0.757	0.831	0.919

The Ridge model provides only a marginal improvement over `sobel_variance`, and only at certain tolerances: it outperforms at 0.10 mm (0.676 vs 0.675) and 0.50 mm (0.831 vs 0.830), but underperforms at 0.05 mm (0.566 vs 0.604) and 0.20 mm (0.735 vs 0.743). Across all four tolerances, the differences are negligible, indicating that a linear combination of the seven metrics provides no meaningful improvement over the best single metric. The MLP, by contrast, consistently and substantially outperforms both: 0.757 at 0.10 mm (8.2 percentage points over the best individual metric), 0.654 at 0.05 mm (5.0 points over `sobel_variance`), and 0.919 at 0.50 mm. This consistent gap across all tolerances strongly suggests that non-linear interactions between the metrics carry meaningful predictive signal that a linear model cannot exploit.

Per-sample-type breakdown. Table 5 shows fraction correct broken down by sample type at `tol = 0.1mm`. The same results are visualised in Figures 5 and 6. The plots make clear that both models perform very well on lillium anther and USAF_lr, while paramecium stained remains the most challenging sample type. The MLP’s advantage over Ridge is visually dominated by its substantially improved performance on paramecium stained samples.

Table 5: Fraction correct by sample type at `tol = 0.1mm`. n denotes the number of test images per type.

Sample type	n	Ridge ($\alpha = 1.0$)	MLP
Bee wing	28	0.643	0.643
Ipomoea leaf	26	0.577	0.615
Lillium anther	28	1.000	1.000
Paramecium stained	28	0.250	0.607
USAF_lr	26	0.923	0.923
Overall	136	0.676	0.757

Both models achieve a perfect fraction correct of 1.000 on lillium anther and 0.923 on USAF_lr, consistent with the clean, well-defined focus curves observed during exploration. Bee wing and ipomoea leaf are intermediate; the MLP matches or exceeds Ridge on both. The most striking difference is on paramecium stained: Ridge achieves only 0.250, barely above random prediction, while the MLP achieves 0.607 — a gap of 0.357. This is the largest inter-model difference observed across any sample type and confirms that non-linear interactions are most important for complex,

three-dimensional biological samples where no single depth plane is uniformly in focus. It is also worth noting that the MLP results carry some run-to-run variability due to random weight initialisation; the values reported here represent a single training run. Future work should repeat training with multiple random seeds and report the mean and standard deviation of performance.

Learned weights (Ridge). The learned weights of the Ridge model at $\alpha = 1.0$ are shown in Figure 2 and Table 6. The two largest-magnitude weights are assigned to `norm_var` (+0.788) and `sum` (−0.746), despite both metrics performing at chance level individually (0.059). Conversely, `sobel_variance`, the best individual metric, receives only −0.083, and `peak` is effectively ignored (−0.004). This counter-intuitive pattern is characteristic of Ridge regression with correlated inputs: the gradient-based metrics (`sobel`, `sobel_variance`, `brenner`) are mutually correlated and so provide diminishing marginal information; the model redistributes weight towards the structurally different `norm_var` and `sum` metrics, which may encode complementary information not captured by the gradient-based metrics. That `norm_var` alone scores 0.059 but receives the largest positive weight illustrates that individual metric performance under `argmin` is not a reliable guide to the incremental value a metric adds within a combination.

Table 6: Learned Ridge regression weights ($\alpha = 1.0$) alongside individual metric performance at `tol = 0.1mm`.

Metric	Individual fraction correct	Ridge weight
<code>sum</code>	0.059	−0.7455
<code>peak</code>	0.228	−0.0038
<code>sobel</code>	0.669	−0.0226
<code>sobel_variance</code>	0.675	−0.0826
<code>brenner</code>	0.626	+0.0546
<code>dark_focus</code>	0.580	−0.1352
<code>norm_var</code>	0.059	+0.7878
Bias	—	+0.1358

Single-image curve inspection. Figures 7 and 8 show the combined metric output alongside the seven raw normalised metric curves for one bee wing test image. In both cases the learned model produces a smoother focus curve than the individual metrics. The MLP output exhibits a clearer minimum near the true depth, consistent with its superior quantitative performance reported in Table 4.

Limitations. The Ridge model provides no meaningful improvement over the best individual metric and is unable to capture non-linear interactions. Both models struggle with paramecium stained samples, though the MLP substantially mitigates this. The ground truth depths were determined manually and carry inherent subjectivity for three-dimensional samples, placing an upper bound on achievable performance. MLP results are subject to run-to-run variability from random initialisation. The filtering step reduced the dataset from 750 to 676 holograms. Future work could investigate alternative target-generation methods that remain valid near reconstruction boundaries, allowing a larger fraction of the available data to be retained. Given more time, training separate models per sample type, running the MLP multiple times to report mean and standard deviation, and exploring larger architectures would be the most productive next steps.

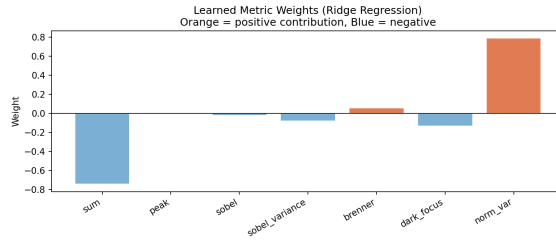


Figure 2: Learned Ridge regression weights for each focus metric. Positive values indicate metrics that increase the predicted focus score, while negative values decrease it.

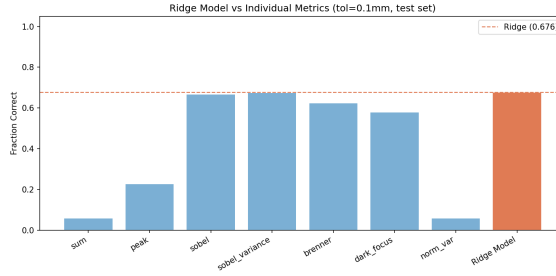


Figure 3: Fraction correct at 0.1 mm tolerance for the Ridge model and all individual focus metrics.

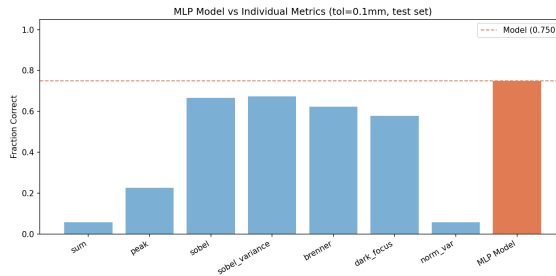


Figure 4: Fraction correct at 0.1 mm tolerance for the MLP model and all individual focus metrics.

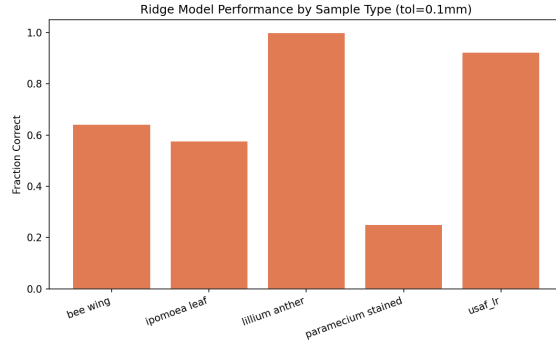


Figure 5: Comparison of the ridge model against each of the 5 sample types

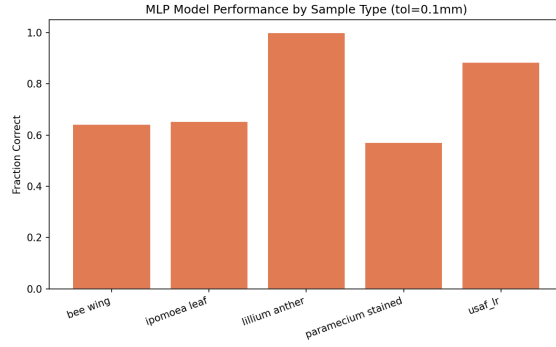


Figure 6: Comparison of the MLP model against each of the 5 sample types

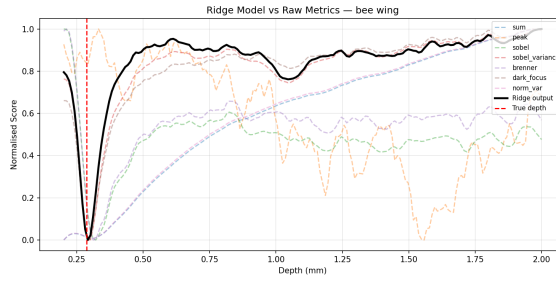


Figure 7: Single-image curve for one individual example, a bee wing - Ridge

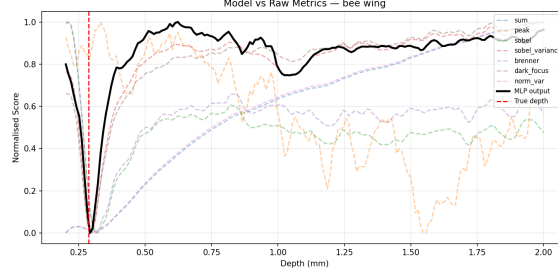


Figure 8: Single-image curve for one individual example, a bee wing - MLP

8 Launch

To use the trained model on new holographic data, the same preprocessing pipeline must be applied in the same order: remove the initial 20 depth values, apply per-image normalisation, apply the same `StandardScaler` (already fit on training data), and then call `model.predict()`. For the Ridge model, the output is negated so that the minimum corresponds to the predicted focus depth, matching the convention used by `fraction_correct`.

Listing 4: Example code for preprocessing and predicting for launch

```
# Preprocessing new data
new_scores_norm = normalise_per_image(new_scores[:, 20:, :])
new_scores_flat = new_scores_norm.reshape(-1, 7)
new_scores_scaled = scaler.transform(new_scores_flat)    # use the already-
               fitted scaler

# Predict and find best focus depth
predictions = model.predict(new_scores_scaled).reshape(num_new_images, -1)
best_focus_indices = np.argmax(predictions, axis=1)    # highest prediction
               = best focus
best_focus_depths = data.depths[best_focus_indices]
```

For continuous monitoring, `fraction_correct` should be evaluated periodically on new labelled images as they become available. If performance degrades (for example when imaging a new sample type or after instrument recalibration) the model should be retrained on an expanded dataset that includes the new data. Because Ridge regression trains in milliseconds, retraining is cheap. The `alpha` and `sigma` hyperparameters should be re-validated whenever the training set changes substantially, as the optimal values may shift with a different data distribution.

References

- [1] Arda Aydinlar. Helper functions for focus metric analysis. <https://github.com/Arda0801/PHYS5014-AI-Project/blob/main/helpers.py>, 2026. Accessed: 2026-06-02.

- [2] Arda Aydinlar. Ridge regression implementation for combining focus metrics. https://github.com/Arda0801/PHYS5014-AI-Project/blob/main/ridge_regression.py, 2026. Accessed: 2026-06-02.
- [3] Aurlien Geron. Machine learning notebooks, 3rd edition. <https://github.com/ageron/handson-ml3>, 2022. Accessed: 2026-06-02.
- [4] Michael Hughes. Pyholoscope: Fast holographic microscopy for python. <https://github.com/MikeHughesKent/PyHoloscope>, 2025. Accessed: 2026-06-02.
- [5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [6] PyTorch Contributors. Build the neural network. https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html, 2026. Accessed: 2026-06-02.
- [7] Scikit-Learn Developers. Ridge. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html, 2026. Accessed: 2026-06-02.
- [8] Scikit-Learn Developers. StandardScaler. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>, 2026. Accessed: 2026-06-02.

Appendix: Supplementary Figures and Code

Listing 5: Python script for Ridge Regression to Combine Focus Metrics in Inline Holographic Microscopy.

```
#Linear regression model for focus prediction. This is a simpler, more interpretable

# --- Imports
-----

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from helpers import load_focus_data, fraction_correct

# --- Load data
-----

# Same as the MLP version. offset=20 removes the noisy first 0.2mm of depths.

print("Loading data...")
data = load_focus_data(offset=20)

print(f"Images: {data.num_images}")
print(f"Depths: {data.num_depths} ({data.depths[0]:.2f}mm to {data.depths[-1]:.2f}mm)")
print(f"Metrics: {data.num_metrics} {data.metrics}")
print(f"Types: {np.unique(data.image_types)}")

# --- Baseline performance of individual metrics
-----

# Before training the model, we determine a baseline performance from individual metrics.
# This creates a baseline for the Ridge model to beat.

print("\n--- Baseline (individual metrics, tol=0.1mm) ---")
tol = 0.1
baseline_scores = {}

for i, metric in enumerate(data.metrics):
    curves = data.scores[:, :, i]
    fc = fraction_correct(curves, data.true_depths, data.depths, tol)
    baseline_scores[metric] = fc
    print(f"{metric}<20s}: {fc:.3f}")

# ---Train-test split
-----
```

```

# Identical to the MLP version      split at image level, stratify by type.

num_images      = data.scores.shape[0]
image_indices   = np.arange(num_images)

train_idx, test_idx = train_test_split(
    image_indices,
    test_size=0.2,
    random_state=42,
    stratify=data.image_types
)

train_scores = data.scores[train_idx]
test_scores  = data.scores[test_idx]
train_depths = data.true_depths[train_idx]
test_depths  = data.true_depths[test_idx]

print(f"\nTrain_images: {len(train_idx)}, Test_images: {len(test_idx)}")

# ---Normalise Inputs
-----
# Same normalisation as the MLP version. Each metric is scaled 0 1 within
# each image so that metrics with large absolute values don't dominate.

def normalise_per_image(scores):
    s = scores.copy()
    s_min = s.min(axis=1, keepdims=True)
    s_max = s.max(axis=1, keepdims=True)
    s = (s - s_min) / (s_max - s_min + 1e-8)
    return s

train_scores_norm = normalise_per_image(train_scores)
test_scores_norm  = normalise_per_image(test_scores)

# ---Build Gaussian Target Model
-----
# This is the key difference from the MLP version. Because sklearn's Ridge
# regression is a standard supervised learning model, it needs a target
# value
# (a label) for every single data point.
#
# Each "data point" here is one (image, depth) pair. The input is 7 metric
# values. The target is a number representing "how in-focus is this depth?"
#
# We use a Gaussian centred on the true depth:
# - At the true depth:      target      1.0 (very in-focus)
# - Far from true depth:    target      0.0 (out of focus)
#
# The model then learns to predict these Gaussian values from the 7 metrics
.

```

```

# At test time, we find the depth where the model's output is highest. That
# is our focus prediction. (Note: unlike the MLP which predicts LOW=focused
# here HIGH=focused because the Gaussian peaks at the true depth. We negate
# the output before passing to fraction_correct which expects LOW=focused.)
#
# sigma controls the width of the Gaussian. A smaller sigma (e.g. 0.05)
# makes
# the target very sharp and precise      only depths very close to the true
# depth
# get a high label. A larger sigma (e.g. 0.2) is more forgiving and easier
# to
# learn, but may produce broader, less precise output curves.

def gaussian_target(depths, true_depth, sigma=0.1):
    return np.exp(-0.5 * ((depths - true_depth) / sigma) ** 2)

def build_dataset(scores_norm, true_depths_mm, depths_array, sigma=0.1):
    num_images, num_depths, num_metrics = scores_norm.shape
    X_list = []
    y_list = []

    for i in range(num_images):
        X_list.append(scores_norm[i])
        y_list.append(gaussian_target(depths_array, true_depths_mm[i],
                                      sigma))
    X = np.vstack(X_list)
    y = np.hstack(y_list)
    return X, y

X_train, y_train = build_dataset(train_scores_norm, train_depths, data.
                                depths, sigma=0.1)
X_test, y_test = build_dataset(test_scores_norm, test_depths, data.
                               depths, sigma=0.1)

print(f"\nTraining set shape: X={X_train.shape}, y={y_train.shape}")
print(f"Test set shape: X={X_test.shape}, y={y_test.shape}")

# ---Ridge Regression
# -----
# Ridge regression is sensitive to the scale of features even after per-
# image
# normalisation, because the regularisation penalty treats all features
# equally.
# StandardScaler rescales each of the 7 features to have mean=0 and std=1
# across all training samples. We fit the scaler ONLY on training data and
# then apply the same transformation to test data, never fit on test data,
# as that would be data leakage.

scaler = StandardScaler()

```

```

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ---Train the Ridge Regression model
-----
# Ridge regression is linear regression with L2 regularisation. This means
# it adds a penalty to the loss function that discourages large weights:
#
# The alpha parameter controls how strong the regularisation is:
# - alpha=0: pure linear regression, no regularisation (can overfit)
# - alpha=1: mild regularisation (good default)
# - alpha=100: strong regularisation (weights are pushed towards zero)
#
# Regularisation helps when the 7 metrics are correlated with each other.
# Without it, the model might assign very large positive weight
# to one metric and equally large negative weight to a correlated one,
# which is numerically unstable and generalises poorly.

print("\nTraining Ridge Regression model...")

model = Ridge(alpha=1.0)
model.fit(X_train_scaled, y_train)

print("Training complete.")
print(f"\nLearned weights (one per metric):")
for metric, weight in zip(data.metrics, model.coef_):
    print(f"_{metric}<20s}:_{weight:+.4f}")
print(f"_{model.intercept_:+.4f}")

# The weights tells how much each metric contributes to the combined score.
# A large positive weight means that metric strongly indicates focus when
# high.
# A near-zero weight means that metric is being mostly ignored.

# --- Generate predictions on the test set
-----
# At test time, we run the model on each (image, depth) pair and reshape
# the
# outputs back into curves of shape (num_images, num_depths).
#
# The model outputs HIGH values where it thinks the image is in focus
# because of the Gaussian target.
# fraction_correct expects LOW values at the true depth, so we negate.

y_pred = model.predict(X_test_scaled)
predicted_curves_pos = y_pred.reshape(len(test_idx), data.num_depths)

predicted_curves = -predicted_curves_pos

```



```

# --- Evaluate performance
-----

fc_model = fraction_correct(predicted_curves, test_depths, data.depths, tol
                             =0.1)
print(f"\n---_Results_(tol=0.1mm)_---")
print(f"_Ridge_model_:_{fc_model:.3f}")
for metric, fc in baseline_scores.items():
    print(f"__{metric:<20s}:_{fc:.3f}")

print("\n---_Model_at_multiple_tolerances_---")
for t in [0.05, 0.1, 0.2, 0.5]:
    fc = fraction_correct(predicted_curves, test_depths, data.depths, tol=t
                           )
    print(f"_tol={t:.2f}mm:_{fc:.3f}")

print("\n---_Per_sample_type_(tol=0.1mm)_---")
for sample_type in np.unique(data.image_types):
    mask = data.image_types[test_idx] == sample_type
    if mask.sum() == 0:
        continue
    fc = fraction_correct(predicted_curves[mask], test_depths[mask], data.
                           depths, tol=0.1)
    print(f"__{sample_type:<20s}:_{fc:.3f}_(n={mask.sum()})")

# --- Plots
-----

# --- Plot 1: Learned Weights Bar Chart
-----

# This is unique to the linear model where you can directly inspect what
# the model
# learned. The MLP has hundreds of weights that are hard to interpret; here
# you have exactly 7, one per metric.

plt.figure(figsize=(9, 4))
colours = ['#E07B54' if w > 0 else '#7BAFD4' for w in model.coef_]
plt.bar(data.metrics, model.coef_, color=colours, edgecolor='white')
plt.axhline(0, color='black', linewidth=0.8)
plt.ylabel("Weight")
plt.title("Learned_Metric_Weights_(Ridge_Regression)\nOrange=_positive_
          contribution,_Blue=_negative")
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.savefig("plot_linear_weights.png", dpi=150)
plt.show()

# --- Plot 2: Model vs Baseline Bar Chart
-----

```

```

labels = list(baseline_scores.keys()) + ["Ridge_Model"]
values = list(baseline_scores.values()) + [fc_model]
colours = ["#7BAFD4"] * len(baseline_scores) + ["#E07B54"]

plt.figure(figsize=(10, 5))
plt.bar(labels, values, color=colours, edgecolor='white')
plt.axhline(y=fc_model, color='#E07B54', linestyle='--', linewidth=1.2,
            label=f"Ridge_{fc_model:.3f}")
plt.ylabel("Fraction_Correct")
plt.title("Ridge_Model vs Individual Metrics (tol=0.1mm, test_set)")
plt.xticks(rotation=30, ha='right')
plt.ylim(0, 1.05)
plt.legend()
plt.tight_layout()
plt.savefig("plot_linear_baseline_comparison.png", dpi=150)
plt.show()

# --- Plot 3: Per-Sample-Type Performance
# -----

sample_types = np.unique(data.image_types)
model_per_type = []
for sample_type in sample_types:
    mask = data.image_types[test_idx] == sample_type
    fc = fraction_correct(predicted_curves[mask], test_depths[mask], data.
                          depths, tol=0.1) if mask.sum() > 0 else 0
    model_per_type.append(fc)

plt.figure(figsize=(8, 5))
plt.bar(sample_types, model_per_type, color="#E07B54", edgecolor='white')
plt.ylabel("Fraction_Correct")
plt.title("Ridge_Model Performance by Sample Type (tol=0.1mm)")
plt.ylim(0, 1.05)
plt.xticks(rotation=20, ha='right')
plt.tight_layout()
plt.savefig("plot_linear_per_type.png", dpi=150)
plt.show()

# --- Plot 4: Single Image Model Curve vs Raw Metrics
# -----

# Same diagnostic plot as the MLP version. The model output curve
# should ideally have a clear minimum near the red true-depth line.

example_idx = 0 # Change this index to see different test images

raw_norm = test_scores_norm[example_idx]
mlp_curve = predicted_curves[example_idx]
mlp_norm = (mlp_curve - mlp_curve.min()) / (mlp_curve.max() - mlp_curve.
min() + 1e-8)

```

```

plt.figure(figsize=(10, 5))
for m in range(raw_norm.shape[1]):
    plt.plot(data.depths, raw_norm[:, m], alpha=0.4, linestyle='--', label=
        data.metrics[m])

plt.plot(data.depths, mlp_norm, color='black', linewidth=2.5, label='Ridge_
output')
plt.axvline(x=test_depths[example_idx], color='red', linestyle='--',
    linewidth=1.5, label='True_depth')
plt.xlabel("Depth_(mm)")
plt.ylabel("Normalised_Score")
plt.title(f"Ridge_Model_vs_Raw_Metrics_    {data.image_types[test_idx[
    example_idx]]}")
plt.legend(loc='upper_right', fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("plot_linear_single_image_curve.png", dpi=150)
plt.show()

# --- Plot 5: Gaussian Targets for a Few Training Images
-----
# This plot is unique to the linear version and shows what the target
# labels
# look like. The soft Gaussian curves the model is trained to predict.
# It helps you understand what the model is actually learning to do.

plt.figure(figsize=(10, 4))
for i in range(5):
    true_depth = train_depths[i]
    target = gaussian_target(data.depths, true_depth, sigma=0.1)
    plt.plot(data.depths, target, label=f"True_depth:{true_depth:.2f}mm")

plt.xlabel("Depth_(mm)")
plt.ylabel("Target_Value")
plt.title("Gaussian_Target_Labels_for_5_Training_Images\n(model_learns_to_
predict_these_curves)")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("plot_linear_gaussian_targets.png", dpi=150)
plt.show()

print("\nAll_plots_saved.")

```

Listing 6: Python script for Multi-Layer Perceptron to Combine Focus Metrics in Inline Holographic Microscopy.

```

# --- Imports
-----

```

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
from helpers import load_focus_data, fraction_correct

# - - Load Data
-----
# offset=20 skips the first 20 depth values (0-0.2mm), which are noisy and
# not
# useful for training.

print("Loading data...")
data = load_focus_data(offset=20)

print(f"Images: {data.num_images}")
print(f"Depths: {data.num_depths} ({data.depths[0]:.2f}mm to {data.depths[-1]:.2f}mm)")
print(f"Metrics: {data.num_metrics} {data.metrics}")
print(f"Types: {np.unique(data.image_types)}")

# - - Baseline Performance
-----
# This checks how well each of the 7 metrics performs on its own. This sets
# the
# target metric to beat. The model should learn to combine the strengths of
# all
# metrics to outperform. fraction_correct() finds the depth where each
# metric
# is lowest (its prediction) and checks whether it falls within 'tol' mm of
# the
# true depth.

print("\n--- Baseline (individual metrics, tol=0.1mm) ---")
tol = 0.1
baseline_scores = {}

for i, metric in enumerate(data.metrics):
    curves = data.scores[:, :, i]
    fc = fraction_correct(curves, data.true_depths, data.depths, tol)
    baseline_scores[metric] = fc
    print(f"{metric}<20s: {fc:.3f}")

# - - Train/Test Split
-----
# IMPORTANT: We split at the IMAGE level, not the depth level. If we split
# by
# depth, the same image could appear in both train and test set, leaking

```

```

    information.
# stratify=data.image_types ensures each sample type (bee wing, paramecium
  etc.)
# is proportionally represented in both sets.

num_images      = data.scores.shape[0]
image_indices   = np.arange(num_images)

train_idx, test_idx = train_test_split(
    image_indices,
    test_size=0.2,
    random_state=42,
    stratify=data.image_types
)

train_scores = data.scores[train_idx]
test_scores  = data.scores[test_idx]
train_depths = data.true_depths[train_idx]
test_depths  = data.true_depths[test_idx]

print(f"\nTrain images: {len(train_idx)}, Test images: {len(test_idx)}")

# -- - Normalisation
# -----
# Each metric has a very different value range. We normalise per-image so
  that
# every metric is scaled 0 1 for that image. This stops metrics with
  large
# absolute values from dominating, and helps the model generalise across
  images
# with different brightness/contrast.

def normalise_per_image(scores):
    s = scores.copy()
    s_min = s.min(axis=1, keepdims=True)
    s_max = s.max(axis=1, keepdims=True)
    s = (s - s_min) / (s_max - s_min + 1e-8) #add small epsilon to avoid
      division by zero
    return s

train_scores_norm = normalise_per_image(train_scores)
test_scores_norm  = normalise_per_image(test_scores)

# Convert to PyTorch tensors for use with the neural network
X_train = torch.tensor(train_scores_norm, dtype=torch.float32)
X_test  = torch.tensor(test_scores_norm, dtype=torch.float32)

# -- - True Depth Indices for Loss Function
# -----
# The loss function needs to know *which index* in the depths array

```

```

    corresponds
# to the true depth for each image (not the mm value itself).

def get_true_depth_indices(true_depths_mm, depths_array):
    indices = [np.argmin(np.abs(depths_array - td)) for td in
                true_depths_mm]
    return np.array(indices)

train_depth_indices = get_true_depth_indices(train_depths, data.depths)
test_depth_indices  = get_true_depth_indices(test_depths,  data.depths)

# Convert to tensor for use in loss function
train_idx_tensor = torch.tensor(train_depth_indices, dtype=torch.long)

# - - Loss Function: Margin Ranking Loss
-----
# We want the model to output a LOW score at the true focus depth and HIGH
# scores everywhere else. This is a ranking/ordering problem.
#
# For each image, for every depth d that is NOT the true depth, we penalise
# the model if the true depth score is not sufficiently lower than other
# scores.
# margin controls how much separation we demand between true and false
# depths.

def focus_loss(outputs, true_depth_indices, margin=0.2):
    total_loss = torch.tensor(0.0, requires_grad=True)

    for i in range(outputs.shape[0]):
        true_score = outputs[i, true_depth_indices[i]]
        all_scores = outputs[i]
        penalties = torch.clamp(margin + true_score - all_scores, min=0.0)
        total_loss = total_loss + torch.mean(penalties)

    return total_loss / outputs.shape[0]

# - - - Neural Network Definition
-----
# Simple MLP (Multi-Layer Perceptron):
#   Input: 7 metric values at a single depth
#   Output: 1 focus score for that depth
#
# The network is applied independently to each depth, then the loss is
# computed
# across all depths for each image. This makes the model depth-agnostic
# it
# only sees the metric values, not the depth itself.

class FocusMLP(nn.Module):
    def __init__(self):

```

```

        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(7, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 1)
        )

    def forward(self, x):
        return self.net(x).squeeze(-1)

# - - - Training Loop
-----

model      = FocusMLP()
optimiser  = torch.optim.Adam(model.parameters(), lr=1e-3)

num_epochs = 150
losses     = []

print("\nTraining MLP...")
for epoch in range(num_epochs):
    model.train()
    optimiser.zero_grad()

    outputs = model(X_train)
    loss     = focus_loss(outputs, train_idx_tensor, margin=0.2)

    loss.backward()
    optimiser.step()

    losses.append(loss.item())

    if epoch % 15 == 0:
        print(f"Epoch {epoch:3d}/{num_epochs} | Loss: {loss.item():.4f}")

print("Training complete.")

# - - - Evaluation
-----

model.eval()
with torch.no_grad():
    predicted_curves = model(X_test).numpy()

fc_model = fraction_correct(predicted_curves, test_depths, data.depths, tol=0.1)
print(f"\n--- Results (tol=0.1mm) ---")
print(f"MLP model: {fc_model:.3f}")

```

```

for metric, fc in baseline_scores.items():
    print(f"_{metric}<20s>:_{fc:.3f}")

print("\n---_Model_at_multiple_tolerances_---")
for t in [0.05, 0.1, 0.2, 0.5]:
    fc = fraction_correct(predicted_curves, test_depths, data.depths, tol=t)
    print(f"_{tol}={t:.2f}mm:_{fc:.3f}")

print("\n---_Per_sample_type_(tol=0.1mm)_---")
for sample_type in np.unique(data.image_types):
    mask = data.image_types[test_idx] == sample_type
    if mask.sum() == 0:
        continue
    fc = fraction_correct(predicted_curves[mask], test_depths[mask], data.
        depths, tol=0.1)
    print(f"_{sample_type}<20s>:_{fc:.3f}_{n}={mask.sum()}")

# - - - Plots
-----

# --- Plot 1: Training Loss Curve
-----
plt.figure(figsize=(8, 4))
plt.plot(losses, color='steelblue')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training_Loss_Over_Time")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("plot_training_loss.png", dpi=150)
plt.show()

# --- Plot 2: Model vs Baseline Bar Chart
-----
labels = list(baseline_scores.keys()) + ["MLP_Model"]
values = list(baseline_scores.values()) + [fc_model]
colours = ["#7BAFD4"] * len(baseline_scores) + ["#E07B54"]

plt.figure(figsize=(10, 5))
bars = plt.bar(labels, values, color=colours, edgecolor='white')
plt.axhline(y=fc_model, color='#E07B54', linestyle='--', linewidth=1.2,
    label=f"Model_{fc_model:.3f}")
plt.ylabel("Fraction_Correct")
plt.title("MLP_Model_vs_Individual_Metrics_(tol=0.1mm,_test_set)")
plt.xticks(rotation=30, ha='right')
plt.ylim(0, 1.05)
plt.legend()
plt.tight_layout()
plt.savefig("plot_baseline_comparison.png", dpi=150)

```



```

plt.show()

# --- Plot 3: Per-Sample-Type Performance
-----
sample_types = np.unique(data.image_types)
model_per_type = []
for sample_type in sample_types:
    mask = data.image_types[test_idx] == sample_type
    fc = fraction_correct(predicted_curves[mask], test_depths[mask], data.
        depths, tol=0.1) if mask.sum() > 0 else 0
    model_per_type.append(fc)

plt.figure(figsize=(8, 5))
plt.bar(sample_types, model_per_type, color="#E07B54", edgecolor='white')
plt.ylabel("Fraction Correct")
plt.title("MLP Model Performance by Sample Type (tol=0.1mm)")
plt.ylim(0, 1.05)
plt.xticks(rotation=20, ha='right')
plt.tight_layout()
plt.savefig("plot_per_type.png", dpi=150)
plt.show()

# --- Plot 4: Single Image Model Curve vs Raw Metrics
-----
# Pick one test image and overlay all raw metrics alongside the model
# output.
# This is the most informative diagnostic plot. You can see whether the
# model
# produces a cleaner, sharper minimum near the true depth.

example_idx = 0 # Change this index to see different test images

raw_norm = test_scores_norm[example_idx]
mlp_curve = predicted_curves[example_idx]
mlp_norm = (mlp_curve - mlp_curve.min()) / (mlp_curve.max() - mlp_curve.
    min() + 1e-8)

plt.figure(figsize=(10, 5))
for m in range(raw_norm.shape[1]):
    plt.plot(data.depths, raw_norm[:, m], alpha=0.4, linestyle='--', label=
        data.metrics[m])

plt.plot(data.depths, mlp_norm, color='black', linewidth=2.5, label='MLP
    output')
plt.axvline(x=test_depths[example_idx], color='red', linestyle='--',
    linewidth=1.5, label='True depth')
plt.xlabel("Depth (mm)")
plt.ylabel("Normalised Score")
plt.title(f"Model vs Raw Metrics {data.image_types[test_idx[example_idx]]}")

```

```
plt.legend(loc='upper_right', fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("plot_single_image_curve.png", dpi=150)
plt.show()

print("\nAll plots saved.")
```